

HelixCode Caddy edge — boot persistence

Revision: 1 **Last modified:** 2026-07-01T19:10:00Z **Scope:** rootless-podman boot mechanism for the Caddy edge container.
Authority: constitution §11.4.6 (no-guessing — values mirrored from the live container), §11.4.10 (secret handling), §11.4.174 (never touch other projects' containers).

Problem

The HelixCode edge (Caddy, rootless podman) runs today as container `deploy_caddy_1`, started by `podman-compose` via `deploy/up.sh` with `restart=unless-stopped`. A podman restart policy is **not** a boot mechanism: after a full host reboot nothing re-creates that container, so the edge does **not** come back. Only the two `systemd --user` services (`helix-auth`, `helix-code-server`) return, because they are enabled and `linger` is on.

This guide closes that gap with a **rootless-podman Quadlet** that `systemd`'s podman generator turns into a `helixcode-caddy.service --user` unit which auto-starts on every login/boot (`linger=yes` for `milosvasic` is already set, so that means at host boot).

What ships

File	Role
<code>deploy/quadlet/helixcode-caddy.container</code>	Source of truth Quadlet unit (tracked). Every value mirrors <code>podman inspect deploy_caddy_1</code> .
<code>scripts/install-edge-boot.sh</code>	Install / arm-boot / start / uninstall helper.
<code>docs/guides/EDGE_BOOT.md</code>	This guide.

The unit reproduces the live container exactly (verified with `podman inspect deploy_caddy_1`):

- **image** `localhost/helixcode-caddy-brotli:2` (Pull=never — local image, no registry)
- **network** `pasta:--map-host-loopback,169.254.1.2`

- **ports** 0.0.0.0:52443:443/tcp, 0.0.0.0:52443:443/udp, 0.0.0.0:52080:80/tcp
- **volumes** ./deploy/Caddyfile:/etc/caddy/Caddyfile:Z, ./deploy/tls:/etc/caddy/tls:Z, deploy_caddy-data:/data, deploy_caddy-config:/config (the same named volumes the live container uses, so ACME/cert state carries over)
- **env** ACME_DNS_API_TOKEN= passthrough (empty today, matching live)

The container is named helixcode-caddy (**not** deploy_caddy_1) so it never collides with the compose-created container that already exists in podman storage. Only one process can bind 52443/52080 at a time — the Quadlet unit is the **boot owner**.

Install / enable (the exact command)

```
# From the repo root:
scripts/install-edge-boot.sh
```

That copies the unit to ~/.config/containers/systemd/helixcode-caddy.container and runs systemctl --user daemon-reload. The unit's [Install] WantedBy=default.target means **daemon-reload alone arms boot start** — a Quadlet-generated unit is *not* enabled with systemctl enable; the [Install] section is how it is wired into default.target.

Equivalent manual form (no script):

```
mkdir -p ~/.config/containers/systemd
cp deploy/quadlet/helixcode-caddy.container ~/.config/containers/
systemd/
systemctl --user daemon-reload
```

By design the install does **not** start the container immediately: while the compose deploy_caddy_1 is live it owns the ports, so a start would clash. On the next reboot the compose container does not return and the Quadlet unit claims the ports cleanly.

Activating now (optional)

To hand over from the running compose edge to the Quadlet unit without a reboot, stop the compose container first (frees the ports), then start the unit:

```
scripts/install-edge-boot.sh --start
# refuses if deploy_caddy_1 is still up
# or, manually, after stopping the compose edge:
systemctl --user start helixcode-caddy.service
```

ACME DNS token (§11.4.10)

The tracked unit ships `ACME_DNS_API_TOKEN=` **empty** — matching the live container and never versioning a secret. `scripts/install-edge-boot.sh` only writes a real token if `deploy/.env` already carries a non-empty one, and then only into an **out-of-repo** Quadlet drop-in `~/.config/containers/systemd/helixcode-caddy.container.d/10-token.conf` (mode `0600`, never echoed, never committed) that merges over the empty default.

Note (§11.4.6): Quadlet's `EnvironmentFile=` does **not** honor systemd's - optional prefix or `%h`, so an env-file seam bakes a bogus `--env-file` path and fails the container at boot. The inline-empty + drop-in-override design above avoids that (proven with `podman-system-generator -dryrun`).

Verify (static, does not start anything)

```
QUADLET_UNIT_DIRS="$PWD/deploy/quadlet" \  
/usr/lib/systemd/system-generators/podman-system-generator --  
user -dryrun  
  
# and, on the generated service:  
tmp=$(mktemp -d)  
QUADLET_UNIT_DIRS="$PWD/deploy/quadlet" \  
/usr/lib/systemd/system-generators/podman-system-generator --  
user "$tmp"  
systemd-analyze --user verify "$tmp/helixcode-caddy.service" #  
exit 0 = clean
```

After a real install: `systemctl --user status helixcode-caddy.service`
and, on the next boot, `podman ps` should show the `helixcode-caddy` container up.

Uninstall / disarm

```
scripts/install-edge-boot.sh --uninstall
```

Stops the service, removes the unit + token drop-in, reloads. The named volumes (`deploy_caddy-data` / `deploy_caddy-config`) and the compose stack are left untouched.